



DOSSIER PROFESSIONNEL (DP)

Nom de naissance ➤ DUPONT
Nom d'usage ➤ Entrez votre nom d'usage ici.
Prénom ➤ Martin
Adresse ➤ 51 rue Hoche, 94200 Ivry-sur-Seine

Titre professionnel visé

Développeur web et web mobile
(dossier d'exemple à adapter je-passe-mon-titre-pro.fr)

MODALITÉ D'ACCÈS :

- ☒ Parcours de formation
- ☐ Validation des Acquis de l'Expérience (VAE)

Présentation du dossier

Le dossier professionnel (DP) constitue un élément du système de validation du titre professionnel. **Ce titre est délivré par le Ministère chargé de l'emploi.**

Le DP appartient au candidat. Il le conserve, l'actualise durant son parcours et le présente **obligatoirement à chaque session d'examen.**

Pour rédiger le DP, le candidat peut être aidé par un formateur ou par un accompagnateur VAE.
Il est consulté par le jury au moment de la session d'examen.

Pour prendre sa décision, le jury dispose :

1. des résultats de la mise en situation professionnelle complétés, éventuellement, du questionnaire professionnel ou de l'entretien professionnel ou de l'entretien technique ou du questionnement à partir de productions.
2. du **Dossier Professionnel** (DP) dans lequel le candidat a consigné les preuves de sa pratique professionnelle
3. des résultats des évaluations passées en cours de formation lorsque le candidat évalué est issu d'un parcours de formation
4. de l'entretien final (dans le cadre de la session titre).

[Arrêté du 22 décembre 2015, relatif aux conditions de délivrance des titres professionnels du ministère chargé de l'Emploi]

Ce dossier comporte :

- pour chaque activité-type du titre visé, un à trois exemples de pratique professionnelle ;
- un tableau à renseigner si le candidat souhaite porter à la connaissance du jury la détention d'un titre, d'un diplôme, d'un certificat de qualification professionnelle (CQP) ou des attestations de formation ;
- une déclaration sur l'honneur à compléter et à signer ;
- des documents illustrant la pratique professionnelle du candidat (facultatif)
- des annexes, si nécessaire.

Pour compléter ce dossier, le candidat dispose d'un site web en accès libre sur le site.

DOSSIER PROFESSIONNEL ^(DP)



<http://travail-emploi.gouv.fr/titres-professionnels>

Sommaire

Exemples de pratique professionnelle

Développer la partie front-end d'une application web ou web mobile sécurisée

p.

- Installer et configurer son environnement de travail en fonction du projet web ou web mobile
- Maquetter des interfaces utilisateur web ou web mobile
- Réaliser des interfaces utilisateur statiques web ou web mobile

p. _____

p. _____

p. _____

Développer la partie back-end d'une application web ou web mobile sécurisée

p.

- Mettre en place une base de données relationnelle
- Développer des composants d'accès aux données SQL et NoSQL
- Développer des composants métier côté serveur

p. _____

p. _____

p. _____

Titres, diplômes, CQP, attestations de formation *(facultatif)*

p. _____

Déclaration sur l'honneur

p. _____

Documents illustrant la pratique professionnelle *(facultatif)*

p. _____

Annexes *(Si le RC le prévoit)*

p. _____

EXEMPLES DE PRATIQUE PROFESSIONNELLE

Activité-type 1

Développer la partie front-end d'une application web ou web mobile sécurisée

Exemple n°1 - *Installer et configurer son environnement de travail en fonction du projet web ou web mobile*

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Quand j'ai commencé à travailler sur mon premier projet de développement en entreprise, la première étape a été de préparer mon environnement de travail. C'était une étape essentielle pour que je puisse être opérationnel et collaborer efficacement avec l'équipe. Le projet sur lequel je travaillais concernait le développement d'une application web destinée à faciliter la gestion des événements pour les entreprises, avec une forte exigence sur la sécurité et la réactivité de l'interface.

Analyse de l'environnement technique défini dans le dossier de conception

Avant toute chose, j'ai pris connaissance du dossier de conception technique fourni par le chef de projet. Ce document précisait les technologies utilisées : l'application devait être développée en React.js pour le front-end, avec un backend en Node.js. Le projet nécessitait également l'utilisation de Docker pour virtualiser l'environnement, et nous devions gérer le contrôle des versions via Git et une plateforme collaborative comme GitHub.

Pour bien comprendre les attentes, j'ai échangé avec le chef de projet pour clarifier certains points, notamment sur les services spécifiques à configurer dans les conteneurs Docker. Je me suis également assuré que je comprenais l'ensemble des outils mentionnés, en consultant leur documentation technique.

Installation et configuration des outils de développement

Mon poste de travail était une machine sous Windows 11, et j'ai configuré un environnement multi-plateforme avec les outils nécessaires. Voici les étapes principales que j'ai suivies :

- Installation des bases

J'ai commencé par installer un éditeur de code adapté. Mon choix s'est porté sur Visual Studio Code (VS Code), qui est léger et extensible. J'y ai ajouté plusieurs extensions comme ESLint pour le contrôle de la qualité du code et Prettier pour le formatage.

Ensuite, j'ai installé Node.js (version LTS) pour exécuter les scripts JavaScript côté serveur et gérer les dépendances via npm.

Pour les tests unitaires, j'ai installé Jest, qui était utilisé par l'équipe.

- Mise en place de Docker



Le projet exigeait un environnement de développement proche de la production. J'ai donc installé Docker Desktop et paramétré des conteneurs pour chaque service :

J'ai récupéré les fichiers docker-compose.yml fournis par l'équipe, qui définissaient les conteneurs nécessaires (base de données PostgreSQL, serveur Node.js, serveur de développement React).

Après avoir vérifié les versions dans le fichier, j'ai exécuté **docker-compose up** pour démarrer l'environnement. Cela m'a permis de travailler avec une base de données préconfigurée et un serveur local simulant la production.

- **Gestion des versions et collaboration**

Pour travailler en équipe, j'ai installé Git et configuré GitHub Desktop pour faciliter les synchronisations avec le dépôt distant. J'ai également configuré un fichier **.gitignore** adapté, pour éviter de pousser des fichiers sensibles comme les clés d'accès ou les fichiers de configuration locaux.

En parallèle, nous utilisons Slack pour la communication et Jira pour suivre les tâches et les tickets liés au projet.

Utilisation de la documentation technique et veille technologique

La documentation technique a joué un rôle important tout au long de cette phase. La plupart des outils utilisés disposaient de guides en anglais, et je me suis familiarisé avec la terminologie propre à chaque technologie. Par exemple :

- Pour configurer Docker, j'ai consulté les documents officiels sur le site de Docker afin de comprendre le rôle des volumes et des réseaux dans notre projet.
- Pour Jest, j'ai lu les guides d'installation pour m'assurer de bien paramétrer les fichiers de test.

J'ai également mis en place une veille technologique pour rester à jour sur les outils que j'utilisais :

- J'ai suivi des blogs comme Smashing Magazine et CSS-Tricks, qui publient des articles sur les évolutions en front-end.
- Pour Docker, je suivais les nouveautés via le site officiel et la newsletter Docker Weekly.

Résolution de problèmes et collaboration

Pendant cette étape, j'ai rencontré quelques défis techniques que j'ai résolus en collaboration avec mes collègues :

- Conflits de versions

Lors de l'installation des dépendances via npm, certains modules entraient en conflit à cause de versions incompatibles. Avec l'aide d'un collègue développeur backend, nous avons modifié le fichier **package.json** pour aligner les versions.

- Problèmes réseau avec Docker



Mon environnement ne communiquait pas correctement avec le conteneur PostgreSQL. Après avoir analysé les logs et cherché dans les forums en ligne, j'ai corrigé les paramètres réseau dans le fichier `docker-compose.yml`.

Ces échanges ont également été une opportunité d'apprendre les bonnes pratiques de mes collègues, comme l'utilisation de commandes Git avancées (rebase, cherry-pick) pour gérer les branches de développement.

Tests de l'environnement

Une fois l'environnement configuré, j'ai vérifié son bon fonctionnement en exécutant des tests :

- J'ai démarré l'application en mode développement via le script `npm start` et vérifié que le front-end communiquait correctement avec le backend.
- Pour m'assurer que la base de données fonctionnait, j'ai lancé des requêtes SQL simples en utilisant l'outil pgAdmin.

J'ai également testé la configuration en équipe, en récupérant et en exécutant le projet sur un autre poste de travail pour garantir que notre documentation interne était claire et que l'environnement pouvait être recréé sans problème.

Rétrospective et amélioration continue

À la fin de cette étape, nous avons organisé une petite réunion avec l'équipe pour discuter des points d'amélioration. Une des suggestions que j'ai apportées concernait l'automatisation : j'ai proposé d'ajouter un script bash qui automatiserait l'installation des dépendances et le démarrage des conteneurs Docker pour les nouveaux développeurs.

Cette expérience m'a permis de développer une méthodologie rigoureuse pour installer et configurer un environnement de travail. J'ai appris à travailler avec des outils modernes comme Docker, Git et VS Code, tout en collaborant efficacement avec mes collègues. C'est une compétence que je réutiliserai à coup sûr dans mes futurs projets.

2. Précisez les moyens utilisés :

DOSSIER PROFESSIONNEL (DP)



Visual Studio Code : Éditeur de code avec extensions (ESLint, Prettier).

Node.js : Exécution des scripts et gestion des dépendances (npm).

Docker Desktop : Conteneurs pour simuler l'environnement de production.

Git et GitHub Desktop : Gestion des versions et collaboration.

Slack : Communication avec l'équipe.

Jira : Suivi des tâches du projet.

pgAdmin : Gestion et requêtes SQL pour PostgreSQL.

Documentation technique : Docker, Jest, React.js (français et anglais).

Veille technologique : Blogs (Smashing Magazine, CSS-Tricks), newsletter Docker Weekly.

3. Avec qui avez-vous travaillé ?

J'ai travaillé principalement avec le chef de projet, qui m'a fourni le dossier de conception et les spécifications techniques. J'ai aussi collaboré avec un développeur backend pour résoudre les conflits de versions des dépendances et configurer la base de données PostgreSQL. L'équipe front-end m'a accompagné dans la configuration des outils comme ESLint et Prettier pour harmoniser notre code. Enfin, le support technique m'a aidé à résoudre un problème réseau lié à Docker. Tous ces échanges m'ont permis d'avancer rapidement et de garantir un environnement fonctionnel.

4. Contexte

Nom de l'entreprise, organisme ou association - **NOM DE L'ENTREPRISE**

Chantier, atelier, service - Développement

Période d'exercice - Du Cliquez ici au Cliquez ici

5. Informations complémentaires (facultatif)

Je souligne l'importance de documenter chaque étape de l'installation et de la configuration de l'environnement. Cela permet :

- **De faciliter l'intégration de nouveaux développeurs** : Un guide interne ou un fichier README détaillant les étapes (installation des outils, exécution des scripts, configuration des conteneurs) peut faire gagner du temps à l'équipe.
- **De garantir la reproductibilité** : En cas de changement de poste de travail ou de mise à jour, un environnement bien documenté assure que tout le monde travaille avec les mêmes configurations.

DOSSIER PROFESSIONNEL ^(DP)

- **De maintenir la sécurité** : Pendant l'installation, il est important de configurer les accès sécurisés aux services (comme les mots de passe pour la base de données) et de les exclure des fichiers de version (via `.gitignore`).

Ces pratiques renforcent la cohérence et la fiabilité du projet.

Activité-type 1

Développer la partie front-end d'une application web ou web mobile sécurisée

Exemple n°2 - Maquetter des interfaces utilisateur web ou web mobile

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Pendant mon expérience en entreprise, j'ai eu l'occasion de travailler sur la création de maquettes pour une application web destinée à la gestion des projets internes. Cette tâche était essentielle pour poser les bases du développement front-end et assurer une expérience utilisateur optimale. Voici comment je m'y suis pris.

Analyse des besoins et préparation

Tout a commencé avec le dossier de conception qui m'a été fourni par le chef de projet. Ce document contenait le recueil des besoins des utilisateurs, les spécifications fonctionnelles et la charte graphique de l'entreprise. J'ai pris le temps de l'analyser en détail, en m'assurant de bien comprendre les attentes. Pour m'aider, j'ai organisé une réunion avec l'équipe design et un développeur backend pour clarifier certains points, comme les contraintes techniques et les éléments de sécurisation à intégrer dès la phase de maquette.

Création des wireframes

J'ai d'abord réalisé des wireframes, des croquis simplifiés des interfaces, pour valider la structure et la disposition des éléments. J'ai utilisé Balsamiq Wireframes pour cette étape, car il permet de concevoir rapidement des maquettes basse-fidélité. Les wireframes m'ont permis de m'assurer que l'enchaînement des pages répondait aux besoins des utilisateurs et aux contraintes d'accessibilité, notamment pour les équipements mobiles.

Conception des maquettes haute-fidélité

Une fois les wireframes validés par le chef de projet et les utilisateurs référents, je suis passé à la réalisation des maquettes haute-fidélité avec Figma. Cet outil m'a permis de travailler efficacement sur les détails visuels en respectant la charte graphique de l'entreprise :

- Couleurs : J'ai utilisé les codes couleurs de la charte (teintes bleu et blanc).
- Typographie : J'ai appliqué la police officielle de l'entreprise, Roboto, en respectant les tailles et hiérarchies définies.
- Boutons et icônes : J'ai intégré des éléments interactifs en cohérence avec les besoins d'accessibilité, comme des contrastes suffisants et des tailles adaptées aux utilisateurs en situation de handicap.

Prise en compte de l'expérience utilisateur (UX)



Lors de la conception, je me suis mis à la place de l'utilisateur final. J'ai imaginé différents scénarios d'utilisation pour m'assurer que les interfaces seraient intuitives. Par exemple :

- Sur mobile, j'ai positionné les boutons principaux à portée de pouce.
- J'ai ajouté des messages d'erreur clairs pour guider l'utilisateur en cas de problème.

Pour évaluer l'efficacité des maquettes, j'ai organisé une session de feedback avec deux membres de l'équipe support client, qui ont l'habitude d'interagir avec les utilisateurs finaux. Leurs retours m'ont permis d'ajuster certains détails, comme la simplification du menu principal.

Modélisation des enchaînements d'interfaces

Une fois les maquettes prêtes, j'ai modélisé les enchaînements des pages avec un schéma interactif dans Figma. J'ai utilisé ses fonctionnalités de prototypage pour simuler la navigation entre les écrans. Cela a été très utile pour présenter le flux de l'application au chef de projet et au client final.

Conformité et législation

J'ai veillé à respecter les réglementations liées à l'accessibilité numérique (RGAA) en testant mes maquettes avec un simulateur de contraste et un lecteur d'écran intégré à l'outil Figma. J'ai aussi pris en compte l'éco-conception en limitant le nombre d'éléments graphiques inutiles pour réduire la consommation de ressources lors du chargement des pages.

Livraison et validation

Une fois les maquettes finalisées, je les ai partagées avec l'équipe front-end via Slack et GitHub. J'ai rédigé un document de présentation qui expliquait les choix de conception, les éléments interactifs et les aspects techniques à prendre en compte lors du développement.

Le chef de projet a validé les maquettes après une dernière réunion de synthèse, et j'ai ensuite transféré les fichiers finaux avec leurs annotations à l'équipe de développement.

2. Précisez les moyens utilisés :

Balsamiq Wireframes : Pour les maquettes basse-fidélité.

Figma : Pour les maquettes haute-fidélité et les prototypes interactifs.

Slack : Pour les échanges avec l'équipe.

GitHub : Pour partager les fichiers et centraliser les retours.

Simulateur de contraste : Pour vérifier l'accessibilité visuelle.

Lecteur d'écran : Pour tester la compatibilité avec les outils d'assistance.

3. Avec qui avez-vous travaillé ?

DOSSIER PROFESSIONNEL ^(DP)



J'ai travaillé en étroite collaboration avec le chef de projet, l'équipe design et deux membres de l'équipe support client. Les échanges réguliers avec le développeur backend m'ont permis d'anticiper les contraintes techniques liées aux maquettes. Cette coopération a été essentielle pour garantir la qualité du travail et respecter les délais.

4. Contexte

Nom de l'entreprise, organisme ou association ➤ **NOM DE L'ENTREPRISE**

Chantier, atelier, service ➤ Développement

Période d'exercice ➤ Du Cliquez ici au Cliquez ici

5. Informations complémentaires *(facultatif)*

Lors de la réalisation des maquettes, il est essentiel de prendre en compte les principes d'éco-conception. Cela passe par des choix simples mais efficaces, comme :

- Réduire l'utilisation d'images lourdes en privilégiant des formats optimisés (WebP, SVG).
- Limiter les animations ou les effets graphiques trop gourmands en ressources.
- Structurer le contenu pour améliorer la lisibilité et réduire les clics inutiles.

Ces actions permettent non seulement d'améliorer la performance des interfaces, mais aussi de réduire leur impact environnemental, tout en offrant une meilleure expérience utilisateur, notamment sur les appareils moins performants.

Activité-type 1

Développer la partie front-end d'une application web ou web mobile sécurisée

Exemple n°3 - Réaliser des interfaces utilisateur statiques web ou web mobile

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Quand j'ai commencé à travailler sur les interfaces utilisateur statiques, j'ai d'abord étudié le dossier de conception et les maquettes fournies par le designer UX/UI de l'équipe. Les maquettes étaient assez détaillées, avec des indications précises sur les couleurs, les polices, les tailles des éléments, et leur positionnement. Mon objectif était de transformer ces maquettes en interfaces fonctionnelles tout en respectant les attentes en termes d'expérience utilisateur et d'accessibilité.

Analyse des besoins

Je me suis d'abord plongé dans le dossier de conception. J'ai pris le temps de bien comprendre les maquettes et de discuter avec le designer UX/UI pour éclaircir certains points, notamment les choix graphiques et les contraintes liées aux utilisateurs en situation de handicap. Par exemple, il fallait prévoir des contrastes suffisants et s'assurer que tous les éléments interactifs soient accessibles au clavier.

Mise en place de l'environnement

Pour commencer à coder, j'ai utilisé Visual Studio Code comme éditeur de code. J'ai installé des extensions comme Prettier pour uniformiser le style de code et Live Server pour tester mes pages en local rapidement. J'ai aussi configuré un environnement Git avec GitLab pour versionner mon travail et collaborer avec mes collègues. On avait un dépôt dédié au front-end, ce qui permettait de bien séparer les responsabilités.

Rédaction du code HTML et CSS

Je me suis attaqué au HTML en suivant une structure sémantique stricte. Par exemple, pour un formulaire, j'ai utilisé les balises `<form>`, `<label>`, et `<input>` en respectant les normes d'accessibilité (attribut `aria-label`, `for`, etc.).

Pour le style, j'ai travaillé avec Sass, ce qui m'a permis d'organiser mon code CSS en plusieurs fichiers et d'utiliser des variables pour les couleurs et les tailles. Ça m'a aidé à maintenir une cohérence visuelle tout au long du projet.

Je me suis aussi assuré que les pages soient responsives en utilisant Flexbox et CSS Grid, en testant les interfaces sur différents formats d'écran grâce aux outils de développement intégrés dans Google Chrome.

Adaptation pour l'éco-conception

Pour respecter les principes d'éco-conception, j'ai optimisé les images en utilisant le format WebP et j'ai réduit leur taille avec l'outil TinyPNG avant de les intégrer.

J'ai aussi fait attention à limiter le nombre de requêtes HTTP en regroupant les fichiers CSS et JavaScript.



Enfin, j'ai mis en place un chargement différé pour les images (lazy loading) afin d'améliorer les performances.

Tests et validations

Une fois l'interface codée, j'ai réalisé un jeu d'essai pour vérifier son bon fonctionnement. J'ai testé chaque élément interactif (boutons, liens, formulaires) et corrigé les bugs au fur et à mesure. J'ai aussi utilisé Lighthouse, l'outil intégré à Chrome, pour évaluer les performances, l'accessibilité et le SEO des pages statiques.

Pour les tests unitaires, j'ai utilisé Jest. Par exemple, j'ai vérifié que mes composants de formulaire renvoyaient bien les données saisies ou que les boutons appelaient correctement les fonctions associées.

Publication

Une fois satisfait de mes tests, j'ai publié les pages statiques sur un serveur web via FileZilla. On utilisait un serveur Apache, et j'ai configuré les permissions pour que seules les pages nécessaires soient visibles publiquement.

J'ai aussi travaillé sur le SEO de ces pages en ajoutant des balises <meta> appropriées, des titres clairs, et des descriptions adaptées au public cible.

Documentation

Enfin, j'ai documenté mon travail en anglais et en français. J'ai créé un fichier README.md dans le dépôt Git pour expliquer comment déployer et maintenir les pages. J'y ai inclus une section sur les bonnes pratiques de sécurité, comme éviter l'injection de code grâce à une validation stricte des entrées utilisateur.

2. Précisez les moyens utilisés :

VS Code : Éditeur de code.

Live Server : Permet de tester les pages en local rapidement.

GitLab : Outil de gestion de version et de collaboration.

HTML sémantique : Structure du code avec des balises adaptées.

Sass : Préprocesseur CSS pour mieux organiser le code.

Flexbox : Technique CSS pour créer des mises en page flexibles.

CSS Grid : Technique CSS pour créer des grilles complexes.

Lighthouse : Outil pour évaluer les performances, l'accessibilité et le SEO.

Jest : Framework de tests unitaires pour le JavaScript.

FileZilla : Client FTP pour transférer les fichiers sur le serveur.

Serveur Apache : Serveur web pour héberger les pages.

DOSSIER PROFESSIONNEL (DP)



3. Avec qui avez-vous travaillé ?

J'ai travaillé avec le designer UX/UI pour clarifier les maquettes et discuter des choix graphiques. J'ai collaboré avec le développeur back-end afin de m'assurer que mes pages statiques s'intègrent bien avec l'API. Enfin, le chef de projet a validé les choix d'éco-conception et les besoins spécifiques des utilisateurs.

4. Contexte

Nom de l'entreprise, organisme ou association ▶ **NOM DE L'ENTREPRISE**

Chantier, atelier, service ▶ Développement

Période d'exercice ▶ Du Cliquez ici au Cliquez ici

5. Informations complémentaires (facultatif)

J'ai également pris en compte les retours réguliers du **chef de projet** sur l'expérience utilisateur et les contraintes techniques. Cela m'a permis de faire des ajustements rapides tout au long du développement, garantissant ainsi que les exigences fonctionnelles et techniques étaient toujours alignées.

Activité-type 2

Développer la partie back-end d'une application web ou web mobile sécurisée

Exemple n°1 - Mettre en place une base de données relationnelle

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre de mon expérience en entreprise, j'ai eu l'occasion de travailler sur la création de la base de données pour un projet de développement d'application web. L'une des tâches qui m'a été attribuée était de mettre en place une base de données relationnelle sécurisée, dans le cadre du développement de la partie back-end d'une application web.

Recueil des besoins et analyse du schéma conceptuel

Tout a commencé par une analyse approfondie du dossier de conception fourni par le chef de projet. Ce dossier comprenait les besoins des utilisateurs ainsi que le schéma conceptuel des données. Le recueil des besoins m'a permis de bien comprendre les attentes fonctionnelles et les données nécessaires pour le projet. Il s'agissait, par exemple, de définir les types de données à collecter, comme les informations utilisateurs, les commandes et les produits. Cela m'a aussi permis de visualiser la structure des données sous forme de diagrammes entité-association, où j'ai pu identifier les différentes entités et leurs relations.

Définition du schéma physique des données

Une fois ces informations rassemblées, la prochaine étape a été de définir le schéma physique des données. Cela consistait à traduire les éléments du schéma conceptuel en une structure physique compatible avec une base de données relationnelle. Pour cela, j'ai utilisé MySQL Workbench pour créer le modèle de la base de données. J'ai respecté les règles de normalisation des bases de données, afin d'éviter la redondance des données et d'assurer une bonne performance des requêtes. Ce modèle précisait la manière dont les données seraient stockées, ainsi que les types de champs pour chaque table (int, varchar, date, etc.).

J'ai également dû respecter les règles de nommage en vigueur dans l'entreprise, ce qui est important pour garantir une certaine uniformité et faciliter la maintenance du projet à long terme. Par exemple, les noms des tables devaient être en snake_case (des mots séparés par des underscores), et chaque colonne devait être clairement identifiée par un nom qui décrivait précisément la donnée qu'elle contenait, comme user_id, order_date, etc.

Sécurité, confidentialité et intégrité des données

Une fois le schéma physique établi, il était essentiel de m'assurer que la sécurité, la confidentialité et l'intégrité des données étaient garanties. Nous travaillions sur un projet qui traitait des informations sensibles, telles que des données personnelles, ce qui nécessitait de respecter les normes de sécurité en vigueur, notamment le RGPD (Règlement Général sur la Protection des Données).

Dans un premier temps, j'ai mis en place des restrictions d'accès sur la base de données. J'ai créé des utilisateurs spécifiques dans le système de gestion de base de données (SGBD) avec des droits d'accès différenciés. Par exemple, les utilisateurs responsables de la gestion des données sensibles avaient



uniquement des droits en lecture, tandis que d'autres utilisateurs, comme ceux chargés de la maintenance, avaient des droits plus étendus, y compris en écriture. J'ai utilisé la commande SQL `CREATE USER` pour cela, en attribuant des rôles spécifiques avec la commande `GRANT`. Cela permettait de limiter l'accès aux seules personnes autorisées et d'assurer que les données ne soient pas altérées de manière indue.

J'ai également pris en compte l'intégrité des données en définissant des contraintes sur les tables. Par exemple, j'ai utilisé des clés primaires (`PRIMARY KEY`) pour chaque entité, et des **clés étrangères** (`FOREIGN KEY`) pour garantir la relation entre les différentes tables, comme entre les tables utilisateurs et commandes. En cas de suppression d'une donnée liée, j'ai mis en place des actions de cascading, ce qui permettait de supprimer ou mettre à jour les données associées automatiquement.

Pour renforcer la sécurité des données sensibles, j'ai aussi utilisé des mécanismes de chiffrement. Par exemple, j'ai implémenté une procédure pour chiffrer les mots de passe des utilisateurs en utilisant `bcrypt`, un algorithme de hachage sécurisé. Cela garantissait que les mots de passe ne soient jamais stockés en clair dans la base de données.

Mise en place de la base de données de tests

Une fois la base de données définie, il a fallu la mettre en place dans un environnement de tests. L'objectif était de pouvoir tester l'application sur une base de données réaliste avant de la déployer en production. J'ai créé une base de données de test en clonant la structure de la base de données de production, mais sans y intégrer de données sensibles.

Pour cela, j'ai utilisé phpMyAdmin, un outil que j'ai trouvé très pratique pour créer des bases de données rapidement. Une fois la base de données créée, j'ai importé le schéma physique dans la base de données de test. Puis, j'ai créé un jeu d'essai complet en ajoutant des données fictives dans la base. Cela m'a permis de m'assurer que les relations entre les tables étaient bien respectées et que la structure fonctionnait comme prévu.

Par exemple, j'ai inséré des données fictives dans les tables utilisateurs et commandes, puis j'ai vérifié que les clés étrangères et les actions de mise à jour/cascade fonctionnaient correctement. J'ai aussi utilisé des outils comme DBeaver pour effectuer des requêtes SQL et vérifier que les résultats étaient cohérents.

Sauvegarde et restauration de la base de données de test

Une autre tâche importante a été la sauvegarde de la base de données de test pour pouvoir la restaurer si nécessaire. J'ai utilisé la commande `mysqldump` pour créer une sauvegarde complète de la base de données. Cela permettait de créer un fichier contenant l'ensemble des structures de la base et des données (sous forme de requêtes SQL).

J'ai ensuite testé la procédure de restauration en utilisant la commande `mysql -u username -p < backup.sql`. Cela m'a permis de m'assurer que la sauvegarde était bien fonctionnelle et qu'en cas de besoin, la base de données pourrait être restaurée sans problème.

Compréhension de la documentation technique

DOSSIER PROFESSIONNEL (DP)



Tout au long de ce projet, j'ai dû me familiariser avec la documentation technique de la base de données. Cela inclut non seulement les manuels des outils utilisés, comme MySQL ou phpMyAdmin, mais aussi des documents techniques en anglais concernant les bonnes pratiques de conception de bases de données et de sécurité.

Cela m'a permis de mieux comprendre la manière dont les bases de données sont structurées dans un contexte professionnel et de m'assurer que je respectais les normes en vigueur. Par exemple, en consultant la documentation officielle de MySQL, j'ai pu approfondir mes connaissances sur les types de données, les index, les transactions et les optimisations des requêtes, ce qui m'a permis d'améliorer les performances de la base de données.

Au cours de cette expérience, j'ai pu développer des compétences clés dans la mise en place d'une base de données relationnelle, en respectant les exigences de sécurité, d'intégrité et de confidentialité des données. J'ai appris à travailler avec des outils de gestion de base de données, à définir des schémas physiques efficaces et à réaliser des tests afin de m'assurer de la solidité de la structure. Tout cela m'a permis de contribuer à la mise en place de solutions back-end robustes et sécurisées pour l'application web en question.

2. Précisez les moyens utilisés :

MySQL Workbench : création et gestion de schémas, requêtes SQL.

phpMyAdmin : gestion de base de données via interface graphique, insertion de données.

DBeaver : exécution de requêtes SQL, vérification des données.

MySQL : système de gestion de base de données.

Mysqldump : sauvegarde et restauration de la base de données.

3. Avec qui avez-vous travaillé ?

J'ai travaillé avec le chef de projet pour m'assurer que la structure de la base de données correspondait aux besoins du projet. Il m'a aussi guidé sur les aspects de sécurité et de confidentialité des données. J'ai échangé régulièrement avec le développeur back-end pour m'assurer que mes choix de base de données étaient compatibles avec l'application et qu'ils répondaient aux critères de performance. Enfin, j'ai collaboré avec l'administrateur de base de données pour valider les droits d'accès et les pratiques de sauvegarde.

4. Contexte

Nom de l'entreprise, organisme ou association - **NOM DE L'ENTREPRISE**

DOSSIER PROFESSIONNEL ^(DP)

Chantier, atelier, service	➤ Développement
Période d'exercice	➤ Du Cliquez ici au Cliquez ici

5. Informations complémentaires *(facultatif)*

Une information complémentaire pourrait être que j'ai régulièrement consulté la documentation technique fournie par l'équipe pour m'assurer que toutes les bonnes pratiques et normes de sécurité étaient respectées. Cette documentation incluait des guides en anglais sur la mise en place de la base de données, la gestion des utilisateurs et les processus de sauvegarde, ce qui m'a permis de mieux comprendre les contraintes techniques du projet.

Activité-type 2

Développer la partie back-end d'une application web ou web mobile sécurisée

Exemple n°2 - Développer des composants d'accès aux données SQL et NoSQL

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Pendant mon expérience en entreprise en tant que développeur web, j'ai eu l'occasion de travailler sur plusieurs projets où j'ai dû développer des composants d'accès aux données SQL et NoSQL pour des applications web sécurisées. L'une de mes principales responsabilités était de coder des traitements permettant de consulter, modifier, créer ou supprimer des données dans des bases de données tout en respectant des normes strictes d'intégrité, de sécurité et de confidentialité.

Analyse du dossier de conception et de la structure de la base de données

Avant de commencer à coder quoi que ce soit, je me suis plongé dans le dossier de conception du projet. Ce dossier incluait le recueil des besoins des utilisateurs, les objectifs fonctionnels et un schéma détaillé de la base de données. Dans un premier temps, j'ai étudié la structure de la base de données SQL qui allait être utilisée dans le projet. La base était constituée de plusieurs tables liées entre elles par des relations de type clé étrangère. Chaque table avait des contraintes d'intégrité que je devais respecter, comme les clés primaires, les contraintes de non-nullité, et les contraintes d'unicité.

Ensuite, j'ai aussi étudié les besoins en matière de base de données NoSQL, car le projet nécessitait aussi de stocker certaines données non structurées dans une base de données de type MongoDB. Ces données étaient principalement des logs et des informations sur les utilisateurs, qui devaient être rapidement accessibles.

Coder les traitements d'accès aux données

Une fois la structure de la base de données bien comprise, j'ai commencé à coder les composants d'accès aux données. Je me suis d'abord concentré sur les requêtes SQL. Pour chaque fonctionnalité de l'application, comme la consultation d'un produit ou l'ajout d'un utilisateur, j'ai créé les requêtes nécessaires pour interagir avec la base de données. Par exemple, pour une consultation, j'ai écrit une requête SELECT qui récupérait les informations d'un utilisateur en fonction de son identifiant. De même, pour la création ou la mise à jour de données, j'ai utilisé des requêtes INSERT INTO ou UPDATE, en veillant à ce que toutes les données envoyées par les utilisateurs soient validées avant d'être insérées.

Pour les accès à la base de données NoSQL, j'ai utilisé MongoDB avec des requêtes en utilisant le langage de requête MongoDB. Par exemple, j'ai codé des fonctions permettant d'ajouter, de mettre à jour ou de supprimer des logs en fonction des événements capturés par l'application.

Gérer l'intégrité des données et la confidentialité



Un aspect important de cette tâche était de m'assurer que toutes les données manipulées dans les composants étaient intègres. Cela signifie que les traitements que j'ai codés devaient éviter toute forme de corruption des données, même en cas d'erreurs ou de conflits d'accès concurrentiels.

Pour ce faire, j'ai implémenté des mécanismes de gestion des transactions dans la base de données SQL. Par exemple, en cas d'opération qui modifiait plusieurs tables, j'ai utilisé des transactions pour m'assurer que l'opération était entièrement réussie avant de valider les changements dans la base de données, sinon elle était annulée pour éviter des modifications partielles. Pour la base NoSQL, bien que MongoDB ne supporte pas les transactions au niveau des collections multiples de manière native, j'ai utilisé des mécanismes de verrouillage pour empêcher les accès concurrentiels non maîtrisés.

En ce qui concerne la confidentialité des données, j'ai pris soin de valider toutes les entrées avant de les enregistrer dans la base de données. J'ai utilisé des bibliothèques de validation comme Joi ou express-validator pour m'assurer que les données envoyées par les utilisateurs respectaient des formats spécifiques (comme des emails ou des numéros de téléphone) et ne contenaient aucune donnée sensible qui pourrait compromettre la sécurité.

Tests unitaires et de sécurité

Une fois les composants d'accès aux données développés, je me suis attaqué à la rédaction des tests unitaires. Pour ce faire, j'ai utilisé Mocha et Chai pour les tests côté serveur en Node.js, ainsi que Jest pour tester des fonctions spécifiques. Ces tests ont permis de vérifier que mes composants de consultation, modification, et suppression fonctionnaient correctement, en renvoyant les bons résultats, en gérant les erreurs, et en respectant les contraintes de sécurité.

J'ai également effectué des tests de sécurité, notamment des tests pour prévenir les attaques par injection SQL. Pour ce faire, j'ai mis en place des mécanismes de validation rigoureuse des entrées utilisateur et utilisé des requêtes paramétrées dans mes interactions avec la base de données. Par exemple, plutôt que de concaténer des chaînes pour construire mes requêtes SQL, j'ai utilisé des requêtes préparées avec des paramètres pour éviter les risques d'injection.

Résolution des problèmes et analyse des résultats des tests

Au fur et à mesure des tests unitaires et de sécurité, j'ai parfois rencontré des dysfonctionnements dans les composants que j'avais développés. Dans ces moments-là, j'ai adopté une démarche structurée pour résoudre les problèmes.

Tout d'abord, j'ai utilisé des outils de débogage comme Postman pour simuler les requêtes API et analyser les réponses. Ensuite, j'ai vérifié les logs des erreurs pour repérer des indices de ce qui n'allait pas. Lorsqu'il y avait un dysfonctionnement avec la base de données, comme des erreurs d'intégrité ou des conflits d'accès, j'ai analysé le schéma des données et les requêtes utilisées pour identifier les causes profondes. Une fois le problème identifié, j'ai corrigé le code et réexécuté les tests.

J'ai aussi utilisé des outils comme **OWASP ZAP** pour analyser les vulnérabilités potentielles dans l'application et vérifier que mes traitements d'accès aux données étaient bien sécurisés contre les attaques courantes.

DOSSIER PROFESSIONNEL (DP)



Veille technologique

Dans le cadre de mes missions, il était important de rester informé des évolutions techniques, surtout en ce qui concerne les bases de données SQL et NoSQL. J'ai donc régulièrement effectué une veille technologique pour suivre les dernières tendances et bonnes pratiques. J'ai consulté des blogs spécialisés, comme ceux de **MongoDB** et **MySQL**, et lu des articles sur des plateformes comme **Dev.to** ou **Medium**. J'ai aussi assisté à des webinars et des meetups pour découvrir de nouvelles techniques de sécurisation des bases de données et de gestion des performances.

2. Précisez les moyens utilisés :

MySQL : gestion de la base de données relationnelle.

MongoDB : gestion de la base de données NoSQL.

Mocha & Chai : tests unitaires.

Jest : tests côté serveur.

Postman : tests d'API.

OWASP ZAP : analyse de la sécurité.

Express-validator et **Joi** : validation des entrées utilisateur.

3. Avec qui avez-vous travaillé ?

J'ai collaboré de manière étroite avec plusieurs collègues durant ce projet. Le chef de projet a supervisé les choix techniques et a validé mes décisions concernant la structure de la base de données et les traitements. Le développeur back-end avec qui je travaillais a veillé à l'intégration des composants d'accès aux données avec le reste de l'application et m'a aidé à résoudre les problèmes liés aux transactions. Enfin, l'administrateur de base de données m'a conseillé sur les meilleures pratiques de gestion des droits d'accès et de sauvegarde des données.

4. Contexte

Nom de l'entreprise, organisme ou association - **NOM DE L'ENTREPRISE**

Chantier, atelier, service - Développement

Période d'exercice - Du Cliquez ici au Cliquez ici

5. Informations complémentaires (facultatif)

DOSSIER PROFESSIONNEL (DP)

Cette expérience m'a permis d'acquérir une solide compréhension des bases de données SQL et NoSQL, des techniques de sécurisation des données et des bonnes pratiques en matière de tests et de validation. J'ai pu mettre en œuvre une approche rigoureuse et structurée pour le développement d'applications web sécurisées, tout en garantissant l'intégrité et la confidentialité des données.

Activité-type 2

Développer la partie back-end d'une application web ou web mobile sécurisée

Exemple n°3 - Développer des composants métier côté serveur

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Lors de mon expérience en entreprise en tant que développeur web, une partie importante de mon travail a été dédiée au développement des composants métiers côté serveur pour des applications web sécurisées. Ce type de développement nécessite à la fois une bonne maîtrise de la programmation orientée objet, une attention particulière à la sécurité des données et un respect des bonnes pratiques de développement. Voici comment je me suis investi dans cette tâche et quelles étapes j'ai suivies pour répondre aux exigences du projet.

Analyse du dossier de conception

Avant de commencer à coder, il est essentiel de comprendre en profondeur les besoins du projet. Le dossier de conception était le point de départ de mon travail. Il contenait une description des fonctionnalités à implémenter ainsi qu'une spécification détaillée de l'architecture de l'application. Ce dossier précisait les interactions avec la base de données, les API à utiliser, et les exigences de sécurité.

J'ai pris le temps de bien analyser chaque fonctionnalité pour être sûr de comprendre le rôle de chaque composant métier à développer côté serveur. Par exemple, pour un projet de gestion des utilisateurs, le dossier de conception précisait que je devais développer une fonction pour gérer l'authentification des utilisateurs, la gestion des rôles et des permissions, ainsi qu'une autre pour le suivi des activités des utilisateurs dans l'application.

Développement des composants métier côté serveur

L'une des premières étapes de mon travail a été de développer les traitements côté serveur. Pour cela, j'ai utilisé le framework Node.js avec Express pour structurer les routes et gérer les requêtes HTTP. Le back-end de l'application était principalement composé de services qui géraient des opérations telles que l'authentification, l'autorisation et la gestion des ressources (par exemple, les utilisateurs, les commandes, etc.).

Pour chaque composant métier, j'ai suivi les principes de la programmation orientée objet (POO), ce qui m'a permis d'assurer une meilleure organisation du code, une réutilisabilité des modules et une maintenance facilitée. Par exemple, pour la gestion des utilisateurs, j'ai créé une classe User avec des méthodes pour ajouter, modifier et supprimer des utilisateurs. J'ai respecté les règles de nommage en suivant la convention camelCase pour les variables et les noms de méthodes, et les classes ont été nommées en PascalCase.

De plus, j'ai développé certains composants en mode asynchrone, en particulier pour les opérations de lecture et d'écriture en base de données, afin d'améliorer les performances de l'application. Pour cela, j'ai utilisé async/await pour gérer les promesses et les requêtes non bloquantes, garantissant ainsi que l'application reste réactive même lors d'opérations longues.



Sécurisation des composants

Un aspect clé de ce travail était de sécuriser les composants développés. Dans un environnement de développement professionnel, il est impératif de prendre en compte la sécurité dès le début du projet. Lors de la conception des composants métiers, j'ai utilisé un style défensif pour anticiper les problèmes de sécurité.

Par exemple, pour la gestion des mots de passe des utilisateurs, j'ai utilisé le module bcryptjs pour le hachage des mots de passe avant de les enregistrer en base de données. J'ai aussi mis en place des mécanismes de validation des données d'entrée à l'aide de express-validator, ce qui permettait de filtrer les données entrantes pour éviter les attaques par injection SQL ou XSS.

J'ai également intégré des tokens JWT pour la gestion de l'authentification basée sur des sessions. Chaque fois qu'un utilisateur se connectait, un token JWT était généré et envoyé au client. Ce token était ensuite utilisé pour vérifier l'identité de l'utilisateur lors de chaque requête ultérieure, garantissant ainsi que les données sensibles étaient protégées.

Documentation du code

Une fois les composants métiers développés, j'ai pris soin de bien documenter mon code. La documentation est essentielle dans un cadre professionnel, notamment pour faciliter la compréhension du code par mes collègues et pour assurer la maintenance à long terme du projet.

J'ai utilisé des commentaires clairs et précis pour expliquer le fonctionnement de chaque méthode et classe, en précisant leur rôle, les paramètres attendus et les valeurs retournées. De plus, j'ai documenté certaines parties du code en **anglais**, comme recommandé par les normes de qualité de l'entreprise, afin que le code soit compréhensible par toute l'équipe, y compris les développeurs étrangers. Par exemple, pour une méthode de validation d'un email, j'ai écrit un commentaire expliquant son fonctionnement :

```
/**
 * Validates the email address format.
 * @param {string} email - The email address to validate.
 * @returns {boolean} - Returns true if the email is valid, false otherwise.
 */
function validateEmail(email) { ... }
```

Vérification de la qualité du code

Pour garantir la qualité du code, j'ai utilisé des outils de contrôle de qualité, comme ESLint. Ce linter m'a permis de vérifier que mon code respectait bien les conventions et bonnes pratiques définies par l'entreprise. Par exemple, il m'a signalé si une variable n'était pas utilisée ou si des erreurs de syntaxe



étaient présentes. J'ai aussi configuré Prettier pour formater automatiquement mon code, ce qui garantit une uniformité dans l'ensemble du projet.

De plus, avant de valider mes composants, j'ai effectué des tests unitaires pour chaque fonctionnalité. J'ai utilisé Mocha et Chai pour réaliser ces tests. Chaque test vérifiait que les composants fonctionnaient comme prévu et que toutes les conditions de réussite et d'échec étaient bien gérées.

Tests de sécurité

Une autre étape importante de mon travail a été de réaliser des tests de sécurité pour vérifier que les composants développés étaient bien protégés contre les menaces courantes. En plus des tests de validation des entrées pour éviter les injections, j'ai utilisé OWASP ZAP pour effectuer des scans de sécurité sur l'application. Cet outil m'a permis de détecter des vulnérabilités potentielles, comme des problèmes de configuration de sécurité, des failles dans la gestion des sessions, ou des points d'entrée non sécurisés.

Lors de l'analyse des résultats des tests, j'ai parfois découvert des problèmes de sécurité ou de fonctionnement dans mon code. J'ai adopté une démarche structurée pour résoudre ces dysfonctionnements. J'ai commencé par identifier la cause du problème, souvent en révisant les logs ou en utilisant des outils de débogage comme Postman pour simuler les requêtes et observer les réponses de l'API. Une fois le problème identifié, j'ai corrigé le code et réexécuté les tests pour m'assurer que tout fonctionnait correctement et que la sécurité était assurée.

En conclusion, le développement des composants métiers côté serveur a été une expérience extrêmement enrichissante. J'ai appris à intégrer des bonnes pratiques de programmation, à sécuriser les applications web et à tester rigoureusement chaque composant pour garantir leur fiabilité. Cette expérience m'a également permis de travailler de manière collaborative avec mes collègues et d'adopter une approche structurée et professionnelle pour résoudre les problèmes rencontrés en cours de développement.

2. Précisez les moyens utilisés :

Node.js / Express : pour développer l'API.

bcryptjs : pour le hachage des mots de passe.

JWT : pour l'authentification basée sur des tokens.

Mocha / Chai : pour les tests unitaires.

ESLint et Prettier : pour le contrôle et la mise en forme du code.

OWASP ZAP : pour les tests de sécurité.

Postman : pour tester les API.

3. Avec qui avez-vous travaillé ?

Tout au long de ce projet, j'ai travaillé en collaboration avec plusieurs membres de l'équipe. J'ai échangé avec le responsable sécurité pour m'assurer que les composants développés respectaient les normes de

DOSSIER PROFESSIONNEL (DP)



sécurité de l'entreprise. En outre, le développeur front-end avec qui je collaborais a intégré mes composants côté serveur avec l'interface utilisateur, et nous avons régulièrement fait des tests d'intégration pour vérifier que l'ensemble fonctionnait correctement.

4. Contexte

Nom de l'entreprise, organisme ou association ➤ **NOM DE L'ENTREPRISE**

Chantier, atelier, service ➤ Développement

Période d'exercice ➤ Du Cliquez ici au Cliquez ici

5. Informations complémentaires (facultatif)

Dans le cadre du développement côté serveur, j'ai mis en place une gestion des erreurs robuste pour garantir que l'application ne se termine pas de manière imprévue en cas de dysfonctionnement. Par exemple, j'ai utilisé un middleware d'Express pour gérer les erreurs globales. Ce middleware interceptait les erreurs non traitées et renvoyait des messages d'erreur clairs au client sans exposer de détails techniques qui pourraient compromettre la sécurité de l'application.

DOSSIER PROFESSIONNEL ^(DP)

Titres, diplômes, CQP, attestations de formation

(facultatif)

Intitulé	Autorité ou organisme	Date
Baccalauréat	Lycée Jules Viette	01/07/2019
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.

DOSSIER PROFESSIONNEL ^(DP)

Déclaration sur l'honneur

Je soussigné(e) MARTIN DUPONT *Dossier d'exemple je passe mon titre pro.fr* ,
déclare sur l'honneur que les renseignements fournis dans ce dossier sont exacts et que je
suis l'auteur(e) des réalisations jointes.

Fait à *Ivry sur Seine*

le *Cliquez ici pour choisir une date*

pour faire valoir ce que de droit.

Signature :

Documents illustrant la pratique professionnelle

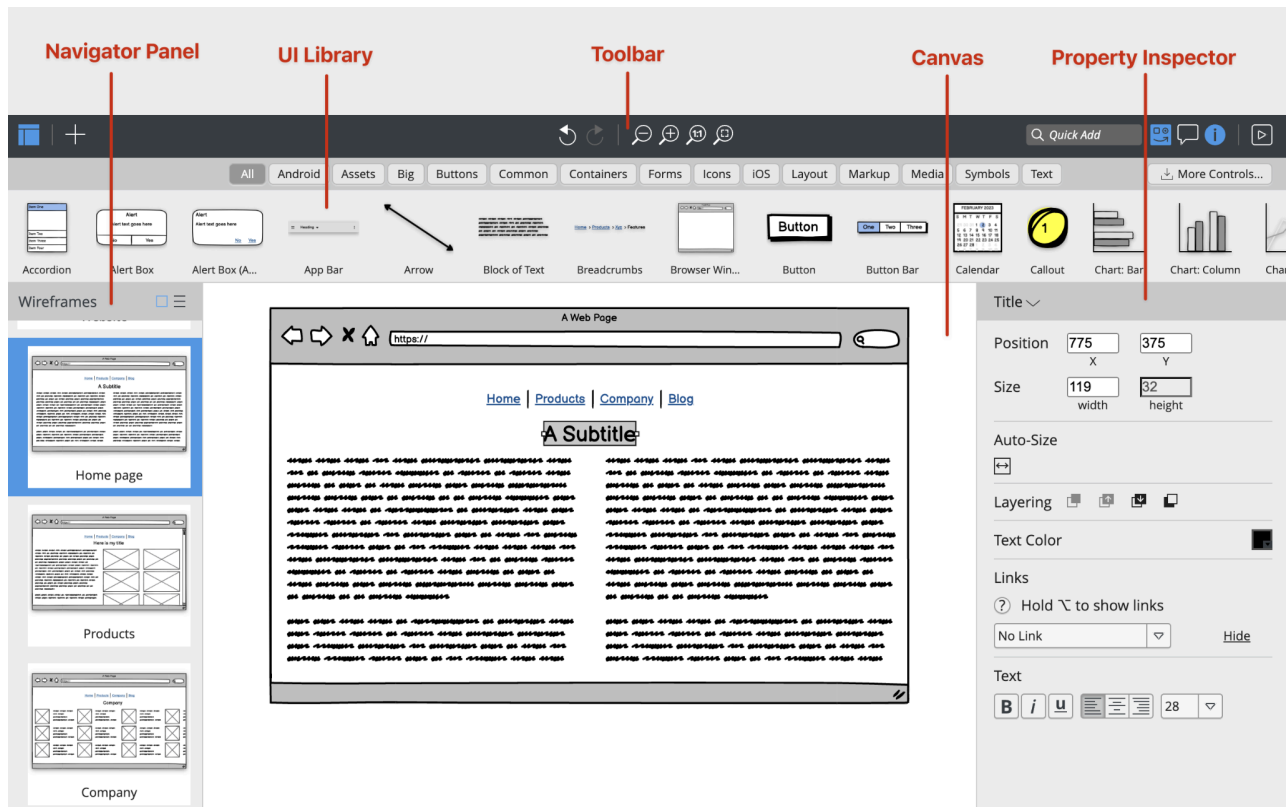
(facultatif)

Intitulé
Annexe 1 : Balsamiq Wireframes, conception rapide de maquettes basse-fidélité
Annexe 2 : Lighthouse, outil de performance de site
Annexe 3 : Visual Studio Code, éditeur de code

DOSSIER PROFESSIONNEL (DP)

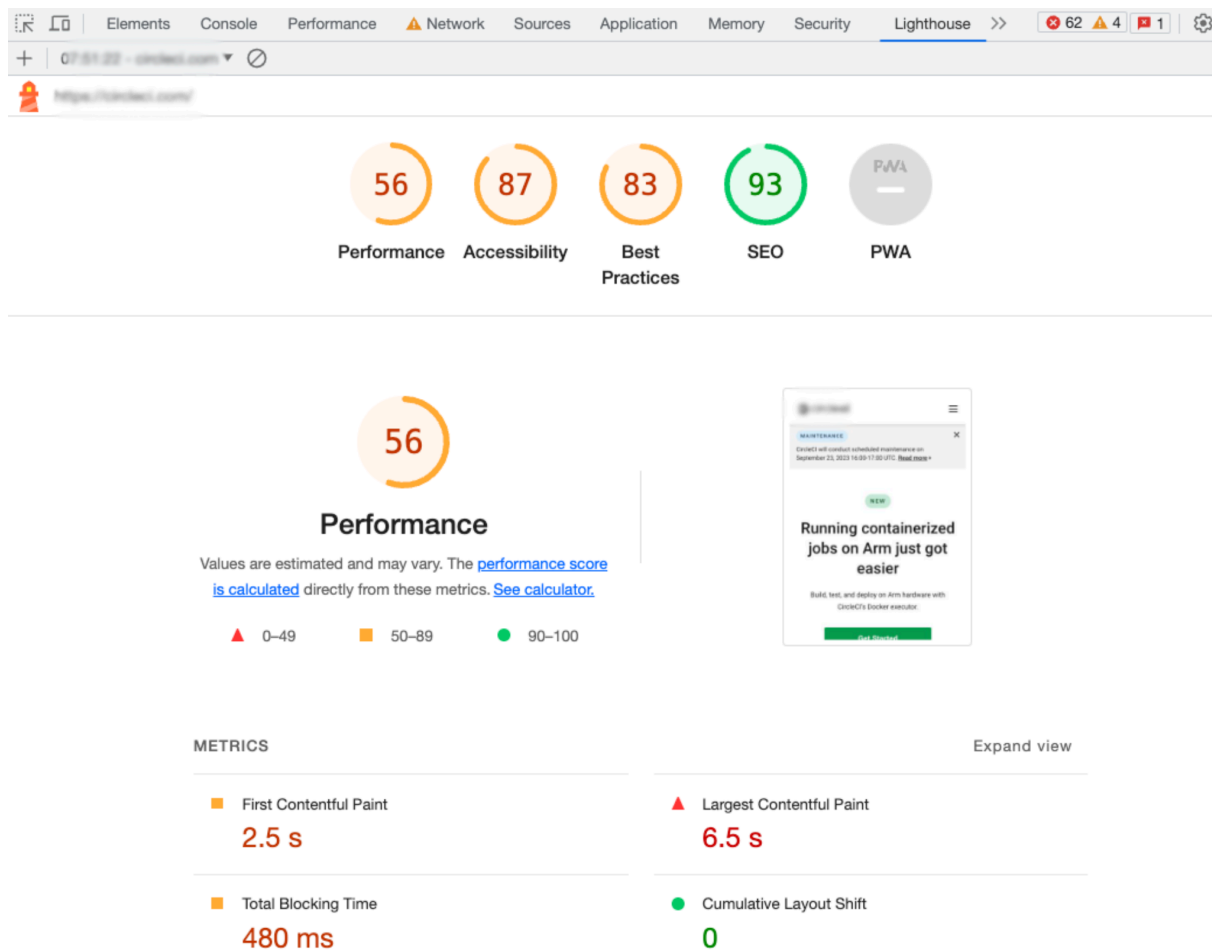
ANNEXES

(Si le RC le prévoit)



Balsamiq Wireframes, conception rapide de maquettes basse-fidélité
[photo d'exemple je-passe-mon-titre-pro.fr à adapter à votre contexte]

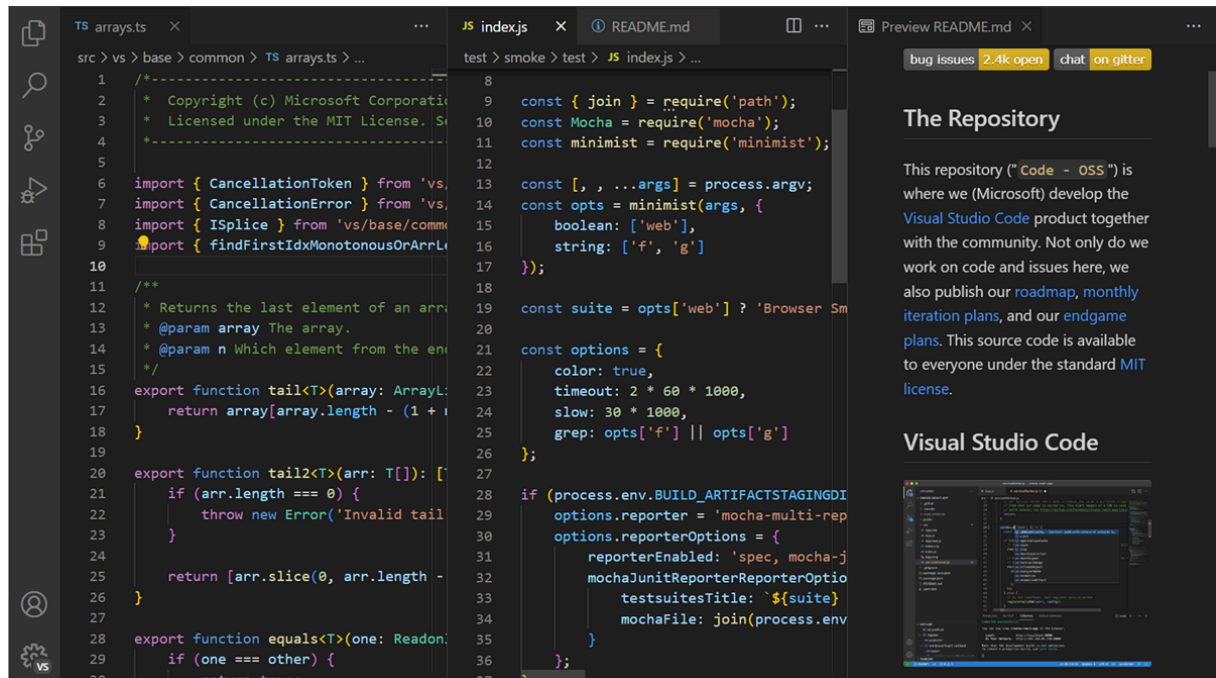
DOSSIER PROFESSIONNEL (DP)



Lighthouse, outil de performance de site

[photo d'exemple je-passe-mon-titre-pro.fr à adapter à votre contexte]

DOSSIER PROFESSIONNEL (DP)



Visual Studio Code, éditeur de code

[photo d'exemple je-passe-mon-titre-pro.fr à adapter à votre contexte]